SAE 1.04

Maxim crepieux

Étape II

A)

```
SELECT *
FROM Clientele
WHERE Clientele.Ville = "Paris"
ORDER BY Clientele.Societe ;
```

J'ai sélectionné tous les clients situés à Paris en filtrant sur leur ville (Paris), puis j'ai trié l'affichage par ordre alphabétique selon le nom de leur société pour faciliter la lecture.

Néanmoins, j'ai remarqué une anomalie : un client est situé dans la ville de "Paris", mais possède un code postal correspondant à un autre département (76). Ne sachant pas si cela était intentionnel, j'ai préféré le signaler.

B)

```
SELECT Produit.NomProd, Produit.CoutAchat, Fournisseur.societe, Fournisseur.Ville
FROM (Produit INNER JOIN Fournisseur ON Produit.NoFour = Fournisseur.NoFour)
INNER JOIN Pays ON Fournisseur.NoPays = Pays.NoPays
WHERE Produit.CoutAchat < 25 AND Fournisseur.NoPays = 7
```

J'ai recherché les produits provenant des États-Unis (avec le code pays adéquat) dont le coût d'achat est inférieur à 25 €. Je fais apparaître leur nom, leur prix ainsi que la société et la ville du fournisseur.

C)

```
SELECT Produit.NomProd, Fournisseur.Societe, Pays.PaysFR, Produit.CoutAchat

FROM ((Produit

INNER JOIN Fournisseur ON Produit.NoFour = Fournisseur.NoFour)

INNER JOIN Categorie ON Produit.CodeCateg = Categorie.CodeCateg)

INNER JOIN Pays ON Fournisseur.NoPays = Pays.NoPays

WHERE (Categorie.NomCateg LIKE "*viande*" OR Categorie.NomCateg LIKE
"*poisson*")

AND Produit.CoutAchat > 50;
```

J'ai extrait les produits appartenant aux catégories de viandes ou poissons dont le coût d'achat dépasse 50 €, tout en affichant leur fournisseur et leur pays d'origine.

D)

```
SELECT DISTINCT Fournisseur.Societe, Categorie.NomCateg FROM ((Fournisseur INNER
JOIN Produit ON Fournisseur.NoFour = Produit.NoFour)

INNER JOIN Categorie ON Produit.CodeCateg = Categorie.CodeCateg)

WHERE Categorie.NomCateg LIKE '*Viande*'

OR Categorie.NomCateg LIKE '*poisson*'

ORDER BY Fournisseur.Societe;
```

J'ai dressé la liste des fournisseurs qui proposent au moins un produit à base de viande ou de poisson, en éliminant les doublons pour garder un résultat propre.

E)

```
SELECT DISTINCT Fournisseur.Societe FROM ((Fournisseur INNER JOIN Produit ON Fournisseur.NoFour = Produit.NoFour)

INNER JOIN Categorie ON Produit.CodeCateg = Categorie.CodeCateg)

WHERE Categorie.NomCateg LIKE '*Viandes*'

AND Fournisseur.NoFour IN ( SELECT Fournisseur.NoFour FROM ((Fournisseur AS Fournisseur

INNER JOIN Produit AS Produit ON Fournisseur.NoFour = Produit.NoFour)

INNER JOIN Categorie AS Categorie ON Produit.CodeCateg = Categorie.CodeCateg)

WHERE Categorie.NomCateg LIKE '*Poissons*' )

 ORDER BY Fournisseur.Societe;
```

J'ai utilisé une sous-requête pour isoler précisément les fournisseurs qui proposent à la fois de la viande et du poisson, en croisant les catalogues de ces deux catégories.

F)

```
SELECT Produit.NomProd, Produit.QteParUnit

FROM Produit
```

```
WHERE Produit.QteParUnit LIKE "*Verre*"

OR Produit.QteParUnit LIKE "*Carton*"

ORDER BY Produit.QteParUnit
```

J'ai interrogé la table des produits pour retrouver ceux qui sont conditionnés dans du carton ou du verre, en triant les résultats par type de conditionnement.

G)

```
SELECT DISTINCT TOP 3 Commande.[N° commande], Commande.DateCde, Clientele.Societe FROM (((Commande INNER JOIN Clientele ON Commande.CodeCli = Clientele.CodeCli)

INNER JOIN DetailCommande ON Commande.[N° commande] = DetailCommande.[N° commande])

INNER JOIN Produit ON DetailCommande.[Réf produit] = Produit.RefProd)

WHERE Clientele.Contact LIKE '*Max Deschamps*'

ORDER BY Commande.DateCde DESC;
```

J'ai affiché les 3 dernières commandes passées par le client Max Deschamps. Pour cela, j'ai relié les commandes aux clients et aux détails, puis j'ai trié les dates du plus récent au plus ancien avec un ordre décroissant.

H)

```
SELECT DISTINCT Fournisseur.Societe

FROM ((Fournisseur INNER JOIN Produit ON Fournisseur.NoFour = Produit.NoFour)

INNER JOIN Categorie ON Produit.CodeCateg = Categorie.CodeCateg)

WHERE Categorie.NomCateg LIKE '*Dessert*'

AND Fournisseur.NoFour NOT IN ( SELECT F.NoFour

FROM ((Fournisseur AS F

INNER JOIN Produit AS P ON F.NoFour = P.NoFour)

INNER JOIN Categorie AS C ON P.CodeCateg = C.CodeCateg)

WHERE C.NomCateg NOT LIKE '*Dessert*' );
```

J'ai mis en place une condition d'exclusion avec une sous-requête pour identifier les fournisseurs dont le catalogue est exclusivement composé de desserts, sans aucun produit d'une autre catégorie.

I)

```
SELECT Fournisseur.Societe , Pays.PaysFR

FROM (Fournisseur LEFT JOIN Produit ON Fournisseur.NoFour = Produit.NoFour)

INNER JOIN Pays ON Fournisseur.NoPays = Pays.NoPays

WHERE Pays.PaysFr <> "France"

AND Produit.RefProd IS NULL

ORDER BY Fournisseur.Societe
```

J'ai utilisé une jointure externe (`LEFT JOIN`) pour repérer tous les fournisseurs situés hors de France qui n'ont actuellement aucun produit rattaché dans le catalogue de KDou.

2)

```
SELECT SUM( DetailCommande.Quantité * Produit.CoutAchat * Categorie.CoefMarge * (1 - DetailCommande.[Remise (%)]) ) AS CA_Total_UE FROM ( ( ( DetailCommande INNER JOIN Produit ON DetailCommande.[Réf produit] = Produit.RefProd ) INNER JOIN Categorie ON Produit.CodeCateg = Categorie.CodeCateg ) INNER JOIN Fournisseur ON Produit.NoFour = Fournisseur.NoFour ) INNER JOIN Pays ON Fournisseur.NoPays = Pays.NoPays WHERE Pays.UE = "Membre";
```

3)

```
SELECT Year(Commande.DateCde) AS Annee, SUM(DetailCommande.Quantité * Produit.CoutAchat * Categorie.CoefMarge) AS CA_Annuel_Sante FROM (((Commande INNER JOIN DetailCommande ON Commande.[N° commande] = DetailCommande.[N° commande]) INNER JOIN Produit ON DetailCommande.[Réf produit] = Produit.RefProd) INNER JOIN Categorie ON Produit.CodeCateg = Categorie.CodeCateg) WHERE Categorie.NomCateg LIKE "*dessert*" OR Categorie.NomCateg LIKE "*boisson*" GROUP BY Year(Commande.DateCde) ORDER BY Year(Commande.DateCde);
```

J'ai calculé le chiffre d'affaires total généré avec les fournisseurs de l'Union Européenne en additionnant (`SUM`) pour chaque ligne le produit de la **`Quantité`**, du coût d'achat, du coefficient de marge de la catégorie et de la remise, j'ai relié en cascade les tables `DetailCommande`, `Produit`, `Categorie`, `Fournisseur` et `Pays` pour lier toutes les données

nécessaires, et j'ai filtré les résultats grâce à la condition sur la colonne UE égale à `"Membre"`.

4)

SELECT COUNT(Produit.RefProd) AS Nb_Produits_Carnes_Non_Commandes FROM (Produit INNER JOIN Categorie ON Produit.CodeCateg = Categorie.CodeCateg) LEFT JOIN DetailCommande ON Produit.RefProd = DetailCommande.[Réf produit] WHERE (Categorie.NomCateg LIKE "*viande*" OR Categorie.NomCateg LIKE "*poisson*") AND DetailCommande.[N° commande] IS NULL;

5)

SELECT Clientele.CodeCli, Clientele.Societe, Clientele.Statut, COUNT(Commande.[N° commande]) AS NbCommandes FROM Clientele INNER JOIN Commande ON Clientele.CodeCli = Commande.CodeCli WHERE Clientele.Statut LIKE "*pilote*" OR Clientele.Statut LIKE "*partenaire*" GROUP BY Clientele.CodeCli, Clientele.Societe, Clientele.Statut ORDER BY COUNT(Commande.[N° commande]) DESC;

Étape III) requête Maxim:

SELECT Produit.NomProd, Fournisseur.Societe, Fournisseur.NoPays

FROM Fournisseur INNER JOIN Produit ON Fournisseur.NoFour = Produit.NoFour

WHERE Fournisseur.NoPays = 9

ORDER BY Produit.NomProd;

Énoncé :

Quels sont les produits du catalogue fournis par nos fournisseurs situés en France ?

Objectif :

Identifier les produits issus de l'approvisionnement local pour réduire l'empreinte carbone liée au transport et valoriser les circuits courts.

Intérêt:

L'intérêt est de privilégier les fournisseurs locaux pour permettre à KDOU un développement durable. Cette requête permet de cibler les offres de proximité afin de minimiser l'impact environnemental du transport logistique et de promouvoir des produits régionaux écoresponsables.

Conception:

J'ai tout d'abord relié la table `Fournisseur` à la table `Produit` via la clé `NoFour`, puis filtré sur les fournisseurs français grâce au champ `NoPays = 9` (correspondant à la France dans la table `Pays`). Enfin, j'ai trié les produits par ordre alphabétique avec `ORDER BY Produit.NomProd`.

//////////////////////////

1-I)

```
SELECT Fournisseur.Societe , Pays.PaysFR

FROM (Fournisseur LEFT JOIN Produit ON Fournisseur.NoFour = Produit.NoFour)
```

```
INNER JOIN Pays ON Fournisseur.NoPays = Pays.NoPays

WHERE Pays.PaysFr <> "France"

AND Produit.RefProd IS NULL

ORDER BY Fournisseur.Societe
```

J'ai utilisé une jointure externe (`LEFT JOIN`) pour repérer tous les fournisseurs situés hors de France qui n'ont actuellement aucun produit rattaché dans le catalogue de KDou.

Pour celui-là, je n'ai pas rencontré de problème particulier, mise à part pour l'orthographe de "différent de " dans le WHERE, qui s'écrit "<>"

1-H)

```
SELECT DISTINCT Fournisseur.Societe

FROM ((Fournisseur INNER JOIN Produit ON Fournisseur.NoFour = Produit.NoFour)

INNER JOIN Categorie ON Produit.CodeCateg = Categorie.CodeCateg)

WHERE Categorie.NomCateg LIKE '*Dessert*'

AND Fournisseur.NoFour NOT IN ( SELECT F.NoFour

FROM ((Fournisseur AS F

INNER JOIN Produit AS P ON F.NoFour = P.NoFour)

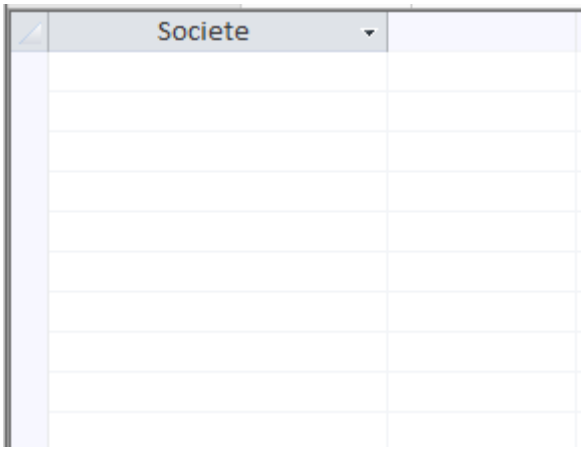
INNER JOIN Categorie AS C ON P.CodeCateg = C.CodeCateg)

WHERE C.NomCateg NOT LIKE '*Dessert*' );
```

J'ai mis en place une condition d'exclusion avec une sous-requête pour identifier les fournisseurs dont le catalogue est exclusivement composé de desserts, sans aucun produit d'une autre catégorie.

la vraie difficulté a été de gérer la logique du "seulement des desserts". C'est facile de chercher des fournisseurs qui vendent des desserts, mais pour être sûr qu'ils ne vendent rien d'autre, il a fallu jouer avec les doubles négations (`NOT IN` et `NOT LIKE`). C'était assez piégeux à construire pour ne pas garder par erreur des fournisseurs qui proposent aussi d'autres types de produits.

Car si on se trompait rien n'apparaissait :



2)

```
SELECT SUM( DetailCommande.Quantité * Produit.CoutAchat * Categorie.CoefMarge * (1 - DetailCommande.[Remise (%)]) ) AS CA_Total_UE FROM ( ( ( DetailCommande INNER JOIN Produit ON DetailCommande.[Réf produit] = Produit.RefProd ) INNER JOIN Categorie ON Produit.CodeCateg = Categorie.CodeCateg ) INNER JOIN Fournisseur ON Produit.NoFour = Fournisseur.NoFour ) INNER JOIN Pays ON Fournisseur.NoPays = Pays.NoPays WHERE Pays.UE = "Membre";
```

J'ai calculé le chiffre d'affaires total généré avec les fournisseurs de l'Union Européenne en additionnant (SUM) pour chaque ligne le produit de la **`Quantité`**, du coût d'achat, du coefficient de marge de la catégorie et de la remise, j'ai relié en cascade les tables `DetailCommande`, `Produit`, `Categorie`, `Fournisseur` et `Pays` pour lier toutes les données nécessaires, et j'ai filtré les résultats grâce à la condition sur la colonne UE égale à `"Membre"`.

La difficulté que j'ai rencontrée est dans la complexité du calcul du prix de vente, qui n'est pas stocké directement sous forme de colonne, ce qui m'a obligé à croiser le coût d'achat du produit avec le coefficient de marge propre à sa catégorie et à intégrer correctement la décote de la remise (sous la forme `1 - [Remise (%)]`) pour obtenir un montant exact.

1-E)

```
SELECT DISTINCT Fournisseur.Societe FROM ((Fournisseur INNER JOIN Produit ON Fournisseur.NoFour = Produit.NoFour)

INNER JOIN Categorie ON Produit.CodeCateg = Categorie.CodeCateg)

WHERE Categorie.NomCateg LIKE '*Viandes*'

AND Fournisseur.NoFour IN ( SELECT Fournisseur.NoFour FROM ((Fournisseur AS Fournisseur

INNER JOIN Produit AS Produit ON Fournisseur.NoFour = Produit.NoFour)

INNER JOIN Categorie AS Categorie ON Produit.CodeCateg = Categorie.CodeCateg)

WHERE Categorie.NomCateg LIKE '*Poissons*' )

 ORDER BY Fournisseur.Societe;
```

J'ai recherché quels fournisseurs proposent à la fois de la viande et du poisson en sélectionnant les sociétés de la table `Fournisseur` via des jointures avec `Produit` et `Categorie`, et j'ai combiné une condition `LIKE 'Viandes'` avec une sous-requête

utilisant un opérateur `IN` filtrée sur `*Poissons*` avant de trier le tout par ordre alphabétique.

Néanmoins, j'ai rencontré une difficulté au niveau des jokers de l'opérateur `LIKE` sous Access, car l'utilisation du symbole % ou de l'astérisque * varie selon la syntaxe choisie, ce qui peut bloquer la reconnaissance des libellés exacts comme `"Poissons et fruits de mer"`.

C)

```
SELECT Produit.NomProd, Fournisseur.Societe, Pays.PaysFR, Produit.CoutAchat
FROM ((Produit
INNER JOIN Fournisseur ON Produit.NoFour = Fournisseur.NoFour)
INNER JOIN Categorie ON Produit.CodeCateg = Categorie.CodeCateg)
INNER JOIN Pays ON Fournisseur.NoPays = Pays.NoPays
WHERE (Categorie.NomCateg LIKE "*viande*" OR Categorie.NomCateg LIKE "*poisson*")
AND Produit.CoutAchat > 50;
```

J'ai sélectionné le nom des produits, la société des fournisseurs, leur pays et leur coût d'achat en reliant en cascade les tables `Produit`, `Fournisseur`, `Categorie` et `Pays`, puis j'ai filtré les résultats pour ne garder que les articles dont la catégorie contient les termes viande ou poisson tout en ayant un coût d'achat supérieur à 50 euros.

Cependant, j'ai buté sur la gestion du filtre textuel `LIKE` dans la clause `WHERE` : il a fallu bien isoler les conditions entre parenthèses avec l'opérateur `OR` et les astérisques (*), afin d'éviter que le filtre sur la catégorie ne fausse le seuil tarifaire des 50 euros.

4)

```
SELECT COUNT(Produit.RefProd) AS Nb_Produits_Carnes_Non_Commandes FROM
(Produit INNER JOIN Categorie ON Produit.CodeCateg = Categorie.CodeCateg) LEFT
JOIN DetailCommande ON Produit.RefProd = DetailCommande.[Réf produit] WHERE
(Categorie.NomCateg LIKE "*viande*" OR Categorie.NomCateg LIKE "*poisson*") AND
DetailCommande.[N° commande] IS NULL;
```

J'ai compté le nombre total de produits en associant la table Produit à sa Categorie, puis en effectuant une jointure externe avec la table DetailCommande, et j'ai filtré l'ensemble pour ne garder que les articles des catégories contenant *viande* ou *poisson* qui n'ont jamais été commandés.

J'ai rencontré plusieurs difficultés pour construire cette requête :

Tout d'abord j'ai eu du mal avec la fonction d'agrégation COUNT que je ne connaissais pas trop au début . J'ai eu aussi du mal à trouver la bonne méthode pour isoler les produits non commandés, avant de comprendre qu'il fallait utiliser une jointure externe (`LEFT JOIN`) combinée avec un filtre `IS NULL` sur le numéro de commande.
J'ai dû faire attention à la syntaxe des joitures multiples, car j'ai dû imbriquer correctement le `INNER JOIN` et le `LEFT JOIN` avec des parenthèses pour éviter des erreurs de structure.
Enfin, j'ai dû penser à encadrer les noms de champs contenant des espaces ou des caractères spéciaux par des crochets, comme `[Réf produit]` et `[N° commande]`, pour que le SGBD ne renvoie pas d'erreur de syntaxe.
Ce qui m'a fait beaucoup échouer, j'ai eu beaucoup de résultats vide et d'erreurs de syntaxe, pour qu'au final je réussise à obtenir un résultat cohérent :

